METODI DI AI PER LA FISICA

S. Giagu - AA 2024/2025

E3 - 19.3.2026

SAPIENZA
UNIVERSITÀ DI ROMA

PANDAS

libreria free-software per manipolare, visualizzare dati tabulati (csv, tab-sv, excel-like) e dati in forma di serie 1D

https://pandas.pydata.org/
http://pandas.pydata.org/pandas-docs/stable/dsintro.html

Tipi di dati di PANDAS

- `Series` represents 1D data, like time series, calendars, the output of one-variable functions, etc.
- `DataFrame` represents 2D data, like a column-separated-values (CSV) file, a microarray, a database table, a matrix, etc.

NOTA: ogni colonna di un DataFrame è una Series, le proprietà dei due tipi sono quindi largamente sovrapposte …

Uso in python:

file csv: 2016.csv

```
Country,Region,Happiness Rank,Happiness Score,Lower Confidence Interval,Upper Confidence Interval,Economy (GDP per Capita),Family,Health (Life Expectancy),Freedom,Trust (Government
Corruption),Generosity,Dystopia Residual
Denmark,Western Europe,1,7.526,7.46,7.592,1.44178,1.16374,0.79504,0.57941,0.44453,0.36171,2.73939
Switzerland,Western Europe,2,7.509,7.428,7.59,1.52733,1.14524,0.86303,0.58557,0.41203,0.28083,2.69463
Iceland,Western Europe,3,7.501,7.333,7.669,1.42666,1.18326,0.86733,0.56624,0.14975,0.47678,2.83137
Norway,Western Europe,4,7.498,7.421,7.575,1.57744,1.1269,0.79579,0.59609,0.35776,0.37895,2.66465
Finland,Western Europe,5,7.413,7.351,7.475,1.40598,1.13464,0.81091,0.57104,0.41004,0.25492,2.82596
Canada,North America,6,7.404,7.335,7.473,1.44015,1.0961,0.8276,0.5737,0.31329,0.44834,2.70485
Netherlands,Western Europe,7,7.339,7.284,7.394,1.46468,1.02912,0.81231,0.55211,0.29927,0.47416,2.70749
New Zealand,Australia and New Zealand,8,7.334,7.264,7.404,1.36066,1.17278,0.83096,0.58147,0.41904,0.49401,2.47553
Australia,Australia and New Zealand,9,7.313,7.241,7.385,1.44443,1.10476,0.8512,0.56837,0.32331,0.47407,2.5465
Sweden,Western Europe,10,7.291,7.227,7.355,1.45181,1.08764,0.83121,0.58218,0.40867,0.38254,2.54734
Israel,Middle East and Northern Africa,11,7.267,7.199,7.335,1.33766,0.99537,0.84917,0.36432,0.08728,0.32288,3.31029
Austria,Western Europe,12,7.119,7.045,7.193,1.45038,1.08383,0.80565,0.54355,0.21348,0.32865,2.69343
United States,North America,13,7.104,7.02,7.188,1.50796,1.04782,0.779,0.48163,0.14868,0.41077,2.72782
Costa Rica,Latin America and Caribbean,14,7.087,6.999,7.175,1.06879,1.02152,0.76146,0.55225,0.10547,0.22553,3.35168
Puerto Rico,Latin America and Caribbean,15,7.039,6.794,7.284,1.35943,1.08113,0.77758,0.46823,0.12275,0.22202,3.0076
Germany,Western Europe,16,6.994,6.93,7.058,1.44787,1.09774,0.81487,0.53466,0.28551,0.30452,2.50931
Brazil,Latin America and Caribbean,17,6.952,6.875,7.029,1.08754,1.03938,0.61415,0.40425,0.14166,0.15776,3.50733
Belgium,Western Europe,18,6.929,6.861,6.997,1.42539,1.05249,0.81959,0.51354,0.26248,0.2424,2.61355
Ireland,Western Europe,19,6.907,6.836,6.978,1.48341,1.16157,0.81455,0.54008,0.29754,0.44963,2.15988
Luxembourg,Western Europe,20,6.871,6.804,6.938,1.69752,1.03999,0.84542,0.5487,0.35329,0.27571,2.11055
Mexico,Latin America and Caribbean,21,6.778,6.68,6.876,1.11508,0.7146,0.71143,0.37709,0.18355,0.11735,3.55906
Singapore,Southeastern Asia,22,6.739,6.674,6.804,1.64555,0.86758,0.94719,0.4877,0.46987,0.32706,1.99375
United Kingdom,Western Europe,23,6.725,6.647,6.803,1.40283,1.08672,0.80991,0.50036,0.27399,0.50156,2.14999
Chile,Latin America and Caribbean,24,6.705,6.615,6.795,1.2167,0.90587,0.81883,0.37789,0.11451,0.31595,2.95505
Panama,Latin America and Caribbean,25,6.701,6.601,6.801,1.18306,0.98912,0.70835,0.48927,0.08423,0.2418,3.00559
Argentina,Latin America and Caribbean,26,6.65,6.56,6.74,1.15137,1.06612,0.69711,0.42284,0.07296,0.10989,3.12985
Czech Republic,Central and Eastern Europe,27,6.596,6.515,6.677,1.30915,1.00793,0.76376,0.41418,0.03986,0.09929,2.96211
United Arab Emirates,Middle East and Northern Africa,28,6.573,6.494,6.652,1.57352,0.87114,0.72993,0.56215,0.35561,0.26591,2.21507
Uruguay,Latin America and Caribbean,29,6.545,6.456,6.634,1.18157,1.03143,0.72183,0.54388,0.21394,0.18056,2.67139
Malta,Western Europe,30,6.488,6.409,6.567,1.30782,1.09879,0.80315,0.54994,0.17554,0.56237,1.99032
Colombia,Latin America and Caribbean,31,6.481,6.384,6.578,1.03032,1.02169,0.59659,0.44735,0.05399,0.15626,3.17471
France,Western Europe,32,6.478,6.397,6.559,1.39488,1.00508,0.83795,0.46562,0.17808,0.1216,2.4744
Thailand,Southeastern Asia,33,6.474,6.396,6.552,1.0893,1.04477,0.64915,0.49553,0.02833,0.58696,2.5796
Saudi Arabia,Middle East and Northern Africa,34,6.379,6.287,6.471,1.48953,0.84829,0.59267,0.37904,0.30008,0.15457,2.61482
Taiwan,Eastern Asia,34,6.379,6.305,6.453,1.39729,0.92624,0.79565,0.32377,0.0663,0.25495,2.61523
Qatar,Middle East and Northern Africa,36,6.375,6.178,6.572,1.82427,0.87964,0.71723,0.56679,0.48049,0.32388,1.58224
Spain,Western Europe,37,6.361,6.288,6.434,1.34253,1.12945,0.87896,0.37545,0.06137,0.17665,2.39663
Algeria,Middle East and Northern Africa,38,6.355,6.227,6.483,1.05266,0.83309,0.61804,0.21006,0.16157,0.07044,3.40904
Guatemala,Latin America and Caribbean,39,6.324,6.213,6.435,0.83454,0.87119,0.54039,0.50379,0.08701,0.28808,3.19863
Suriname,Latin America and Caribbean,40,6.269,6.073,6.465,1.09686,0.77866,0.50933,0.52234,0.12692,0.16665,3.06852
Kuwait,Middle East and Northern Africa,41,6.239,6.154,6.324,1.61714,0.87758,0.63569,0.43166,0.23669,0.15965,2.28085
Bahrain,Middle East and Northern Africa,42,6.218,6.128,6.308,1.44024,0.94397,0.65696,0.47375,0.25772,0.17147,2.27405
Trinidad and Tobago,Latin America and Caribbean,43,6.168,5.95,6.386,1.32572,0.98569,0.52608,0.48453,0.01241,0.31935,2.51394
Venezuela,Latin America and Caribbean,44,6.084,5.973,6.195,1.13367,1.03302,0.61904,0.19847,0.08304,0.0425,2.97468
Slovakia,Central and Eastern Europe,45,6.078,5.996,6.16,1.27973,1.08268,0.70367,0.23391,0.02947,0.13837,2.61065
```

leggiamo il dataset in python:

```
#leggiamo il dataset con pandas

import numpy as np
import pandas as pd

!wget http://giagu.web.cern.ch/giagu/CERN/2016.csv

data = pd.read_csv('2016.csv')
data
```

legge un file .csv e lo inserisce in un DataFrame pandas

	Country	Region	Happiness Rank	Happiness Score	Lower Confidence Interval	Upper Confidence Interval	Economy (GDP per Capita)	Family	Health (Life Expectancy)	Freedom	Trust (Government Corruption)	Generosity	Dystopia Residual
0	Denmark	Western Europe	1	7.526	7.460	7.592	1.44178	1.16374	0.79504	0.57941	0.44453	0.36171	2.73939
1	Switzerland	Western Europe	2	7.509	7.428	7.590	1.52733	1.14524	0.86303	0.58557	0.41203	0.28083	2.69463
2	Iceland	Western Europe	3	7.501	7.333	7.669	1.42666	1.18326	0.86733	0.56624	0.14975	0.47678	2.83137
3	Norway	Western Europe	4	7.498	7.421	7.575	1.57744	1.12690	0.79579	0.59609	0.35776	0.37895	2.66465
4	Finland	Western Europe	5	7.413	7.351	7.475	1.40598	1.13464	0.81091	0.57104	0.41004	0.25492	2.82596
...	...	...	...	...	...	...	...	...	...	...	...	...	...
152	Benin	Sub-Saharan Africa	153	3.484	3.404	3.564	0.39499	0.10419	0.21028	0.39747	0.06681	0.20180	2.10812
153	Afghanistan	Southern Asia	154	3.360	3.288	3.432	0.38227	0.11037	0.17344	0.16430	0.07112	0.31268	2.14558
154	Togo	Sub-Saharan Africa	155	3.303	3.192	3.414	0.28123	0.00000	0.24811	0.34678	0.11587	0.17517	2.13540
155	Syria	Middle East and Northern Africa	156	3.069	2.936	3.202	0.74719	0.14866	0.62994	0.06912	0.17233	0.48397	0.81789
156	Burundi	Sub-Saharan Africa	157	2.905	2.732	3.078	0.06831	0.23442	0.15747	0.04320	0.09419	0.20290	2.10404

157 rows × 13 columns

selezione delle righe in base a condizioni sulle feature:

```
data_selected = data[data['Happiness Rank']<=5]
data_selected
```

	Country	Region	Happiness Rank	Happiness Score	Lower Confidence Interval	Upper Confidence Interval	Economy (GDP per Capita)	Family	Health (Life Expectancy)	Freedom	Trust (Government Corruption)	Generosity	Dystopia Residual
0	Denmark	Western Europe	1	7.526	7.460	7.592	1.44178	1.16374	0.79504	0.57941	0.44453	0.36171	2.73939
1	Switzerland	Western Europe	2	7.509	7.428	7.590	1.52733	1.14524	0.86303	0.58557	0.41203	0.28083	2.69463
2	Iceland	Western Europe	3	7.501	7.333	7.669	1.42666	1.18326	0.86733	0.56624	0.14975	0.47678	2.83137
3	Norway	Western Europe	4	7.498	7.421	7.575	1.57744	1.12690	0.79579	0.59609	0.35776	0.37895	2.66465
4	Finland	Western Europe	5	7.413	7.351	7.475	1.40598	1.13464	0.81091	0.57104	0.41004	0.25492	2.82596

Operation	Syntax	Result
Select column	df[col]	Series
Select multiple columns	df[[col1,col2]]	DataFrame
Select row by label	df.loc[label]	Series
Select row by integer location	df.iloc[loc]	Series
Slice rows	df[5:10]	DataFrame
Select rows by boolean vector	df[bool_vec]	DataFrame

```
data['Happiness Rank']
```

```
0        1
1        2
2        3
3        4
4        5
      ...
152    153
153    154
154    155
155    156
156    157
Name: Happiness Rank, Length:
```

```
data.iloc[3]
```

```
Country                          Norway
Region                   Western Europe
Happiness Rank                        4
Happiness Score                   7.498
Lower Confidence Interval         7.421
Upper Confidence Interval         7.575
Economy (GDP per Capita)        1.57744
Family                           1.1269
Health (Life Expectancy)        0.79579
Freedom                         0.59609
Trust (Government Corruption)   0.35776
Generosity                      0.37895
Dystopia Residual               2.66465
Name: 3, dtype: object
```

statistica descrittiva del campione:

```
# summary statistic del dataset

sel_features = ['Economy (GDP per Capita)','Family','Health (Life Expectancy)','Freedom','Trust (Government Corruption)','Generosity','Dystopia Residual']

data[sel_features].describe()
```

	Economy (GDP per Capita)	Family	Health (Life Expectancy)	Freedom	Trust (Government Corruption)	Generosity	Dystopia Residual
count	157.000000	157.000000	157.000000	157.000000	157.000000	157.000000	157.000000
mean	0.953880	0.793621	0.557619	0.370994	0.137624	0.242635	2.325807
std	0.412595	0.266706	0.229349	0.145507	0.111038	0.133756	0.542220
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.817890
25%	0.670240	0.641840	0.382910	0.257480	0.061260	0.154570	2.031710
50%	1.027800	0.841420	0.596590	0.397470	0.105470	0.222450	2.290740
75%	1.279640	1.021520	0.729930	0.484530	0.175540	0.311850	2.664650
max	1.824270	1.183260	0.952770	0.608480	0.505210	0.819710	3.837720

```
# controllo sulla correlazione dei dati

data[sel_features].corr()
```

	Economy (GDP per Capita)	Family	Health (Life Expectancy)	Freedom	Trust (Government Corruption)	Generosity	Dystopia Residual
Economy (GDP per Capita)	1.000000	0.669540	0.837067	0.362283	0.294185	-0.025531	0.068624
Family	0.669540	1.000000	0.588377	0.450208	0.213561	0.089629	0.119672
Health (Life Expectancy)	0.837067	0.588377	1.000000	0.341199	0.249583	0.075987	0.100857
Freedom	0.362283	0.450208	0.341199	1.000000	0.502054	0.361751	0.091631
Trust (Government Corruption)	0.294185	0.213561	0.249583	0.502054	1.000000	0.305930	-0.002909
Generosity	-0.025531	0.089629	0.075987	0.361751	0.305930	1.000000	-0.133001
Dystopia Residual	0.068624	0.119672	0.100857	0.091631	-0.002909	-0.133001	1.000000

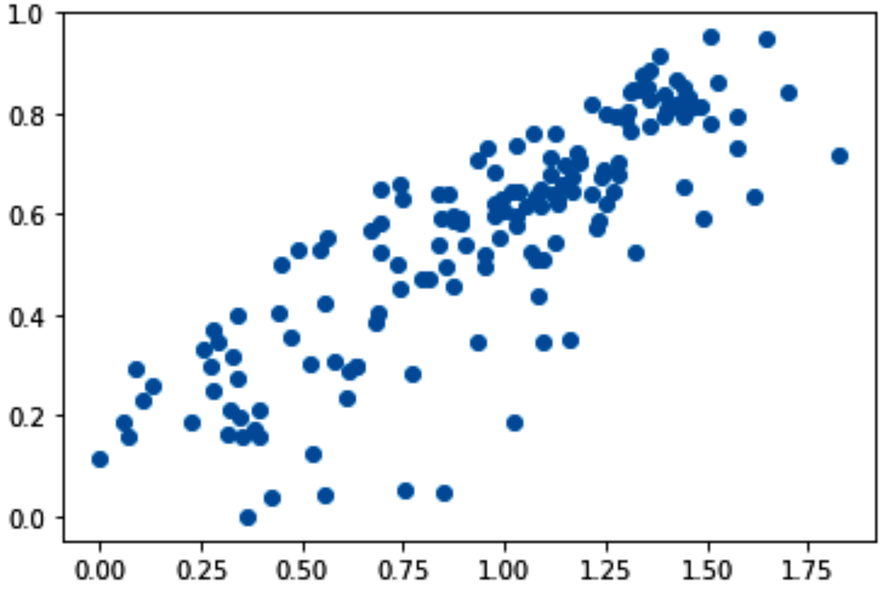
```
data['Happiness Rank'].mean()
```

```
78.98089171974522
```

plotting:

```
plt.scatter(data['Economy (GDP per Capita)'], data['Health (Life Expectancy)'])
```

```
<matplotlib.collections.PathCollection at 0x7f58994ebd50>
```



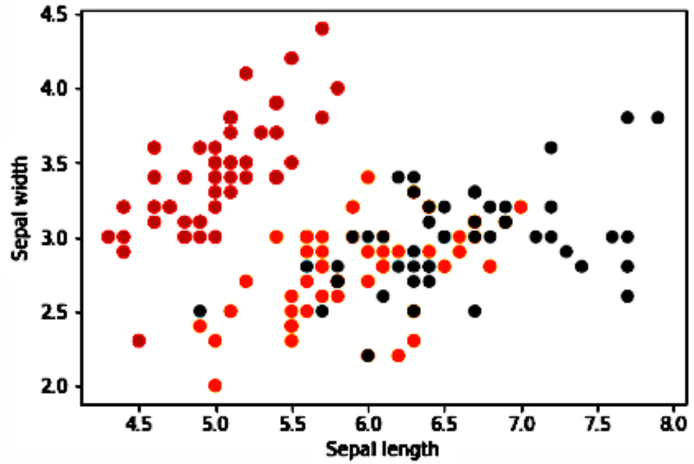
SCIKIT-LEARN

- Scikit-learn è una libreria free-software per il machine learning ottimizzata per il linguaggio python
  - implementa tutti gli algoritmi di ML tradizionale + un grandissimo numero di tools utili per pre-processing e gestione dei dati
  - progettata per interagire con le librerie numeriche e scientifiche di python come numpy
  - è la libreria più usata per il ML in assoluto (i.e. il 90% di chi lavora con il ML la usa)
- web site e documentazione (user-guide scritte molto bene): https://scikit-learn.org/stable/

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn import datasets
```

```
iris = datasets.load_iris() #benchmark dataset iris (3 classi per 4 feature)
X = iris.data[:, :2]  # consideriamo le prime due feature
y = iris.target

# Plot the training points
plt.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.Set1, edgecolor='face')
plt.xlabel('Sepal length')
plt.ylabel('Sepal width')
```

```
Text(0, 0.5, 'Sepal width')
```



CLASSIFICATORE NaiveBayes IN SCIKIT-LEARN

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.naive_bayes import GaussianNB
from sklearn.datasets import make_moons, make_circles, make_classification
from matplotlib.colors import ListedColormap
from sklearn.model_selection import train_test_split
```

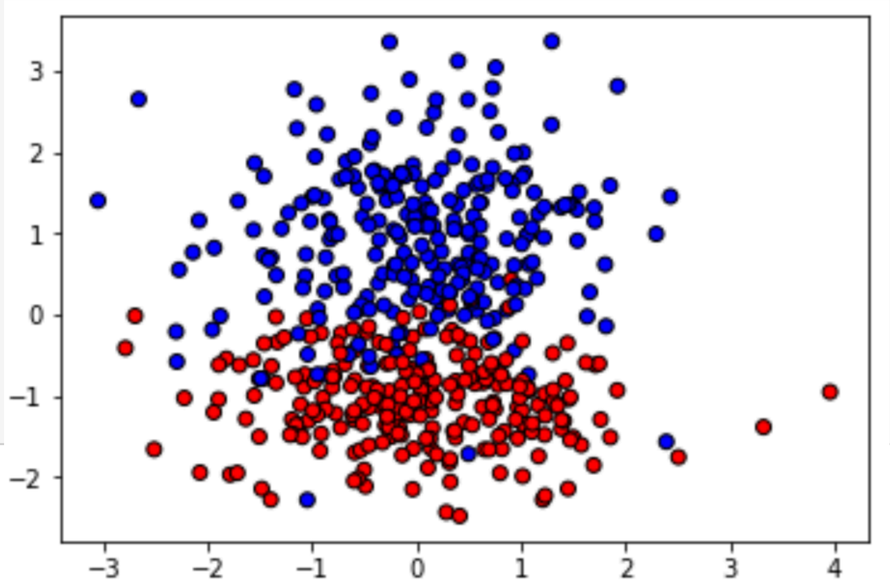
Generazione di un campione TOY

```
# create a toy dataset X data, y target (label)
# redundant features: generated as random linear combinations of the informative features
X, y = make_classification(n_samples=500, n_features=2, n_redundant=0, n_informative=1,
                           random_state=1, n_classes=2, n_clusters_per_class=1, class_sep=1.0)
```

```
print (X[0])
print(y[0])

print (X.shape)
print (y.shape)
```

```
[-0.9612636   2.58773317]
1
(500, 2)
(500,)
```

```
cm_bright = ListedColormap(['#FF0000', '#0000FF'])
plt.figure()
plt.scatter(X[:, 0], X[:, 1], c=y, cmap=cm_bright, edgecolors='k')
plt.show()
```



```
X_train, X_test, y_train, y_test = \
        train_test_split(X, y, test_size=.4, random_state=21, shuffle=True)

print (X_train.shape)
print (X_test.shape)
```

```
(300, 2)
(200, 2)
```

divide il campione in training e test

```
model = GaussianNB()
model.fit(X_train, y_train)
score_train = model.score(X_train, y_train)
score = model.score(X_test, y_test)

print('train accuracy:', score_train)
print('test accuracy:', score)
```

```
train accuracy: 0.9233333333333333
test accuracy: 0.915
```

GaussianNB Model:

la likelihood delle feature è assunta Gaussiana

$$P(y \mid x_1, \ldots, x_n) \propto P(y) \prod_{i=1}^{n} P(x_i \mid y)$$

$$\Downarrow$$

$$\hat{y} = \arg\max_y P(y) \prod_{i=1}^{n} P(x_i \mid y),$$

$$P(x_i \mid y) = \frac{1}{\sqrt{2\pi\sigma_y^2}} \exp\left(-\frac{(x_i - \mu_y)^2}{2\sigma_y^2}\right)$$

- definizione del modello: model = GaussianNB()
- fit del modello ai dati (training): model.fit(X,y)
- misura delle prestazioni (average accuracy): model.score(X,y)

equivalente a fare:

```
accuracy = (np.size(y_test) - np.count_nonzero(predicted-y_test)) / np.size(y_test)
print ('accuracy: ', accuracy)
```

```
accuracy:  0.915
```

- uso del modello addestrato per predizioni (classificazione): model.predict(X)

```
# predict output
predicted = model.predict(X_test)

print('predicted: ', predicted)
print('target: ', y_test)
```

```
predicted:  [1 0 0 0 1 0 1 0 1 1 1 0 0 1 0 0 0 0 1 1 0 0 0 1 0 0 0 1 1 0 0 0 1 1 0 1 0
 0 0 1 0 1 0 1 1 1 1 1 0 1 0 1 0 1 1 1 1 0 0 0 0 1 1 0 1 0 1 1 0 0 1 1 1 0
 1 0 0 1 0 0 0 0 1 1 1 1 0 0 1 0 0 1 0 0 0 1 1 0 1 1 0 1 0 0 0 1 0 0 1 1 0
 0 0 0 1 1 1 0 0 1 1 0 1 0 0 1 1 1 0 1 1 0 0 1 1 0 0 1 0 1 1 1 1 1 0 0 1 1
 1 1 0 1 1 1 1 0 0 0 1 1 1 0 0 0 1 1 0 0 1 1 0 1 0 0 0 0 0 0 1 1 0 1 0 0 0
 0 0 0 0 0 0 1 0 1 1 0 1 0 0 1]
target:  [1 0 0 0 1 0 1 0 1 1 1 0 0 1 0 0 0 1 1 1 0 0 0 1 0 0 0 1 1 0 0 0 1 1 0 1 0
 0 0 1 0 1 0 1 1 1 1 1 0 1 0 1 0 1 1 1 1 0 0 0 1 1 1 0 1 0 1 1 1 0 1 1 1 0
 1 0 0 1 1 0 0 0 0 1 1 1 0 0 1 0 1 1 1 0 0 1 1 0 1 1 0 1 1 0 0 1 0 0 1 0 0
 0 0 0 1 1 1 0 0 1 1 0 0 0 0 1 1 1 0 1 1 0 0 1 1 0 0 1 0 1 1 1 0 1 0 0 1 1
 1 1 0 1 1 1 0 0 0 0 1 1 1 0 0 0 1 0 0 0 1 0 0 1 0 1 0 0 0 0 1 0 0 1 1 0 0
 0 0 0 0 0 0 1 0 1 1 0 1 0 0 1]
```

CLASSIFICATORE k-NN IN SCIKIT-LEARN

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap
from sklearn import neighbors, datasets
from sklearn.model_selection import train_test_split

# import example data: iris dataset
# consists of 3 different types of irises' (Setosa, Versicolour, and Virginica) petal and sepal
# length, stored in a 150x4 numpy.ndarray
# rows being the samples and the columns being: Sepal Length, Sepal Width, Petal Length and Petal Width.
iris = datasets.load_iris()

# we only take the first two features
X = iris.data[:, :2]
y = iris.target

print (X[0])
print(y[0])

print (X.size)

#split sample in training and test
X_train, X_test, y_train, y_test = train_test_split(X,y, test_size=0.2, shuffle=True, random_state=1234)
print(X_train.shape)
print(X_test.shape)
print(y_train.shape)
print(y_test.shape)
```

IRIS dataset disponibile in scikit-learn

```
[5.1 3.5]
0
300
(120, 2)
(30, 2)
(120,)
(30,)
```

è il parametro k

```
n_neighbors = 10

# we create an instance of Neighbours Classifier and fit the data.
clf = neighbors.KNeighborsClassifier(n_neighbors)
clf.fit(X_train, y_train)

print('Accuracy sul traing set: ', clf.score(X_train,y_train))
print('Accuracy sul test set: ', clf.score(X_test,y_test))
```

definisco il classificatore e faccio il training

testo le prestazioni

```
Accuracy sul traing set:  0.8083333333333333
Accuracy sul test set:  0.7666666666666667
```

```
# predict output

predicted = clf.predict(X_test)

print('predicted: ', predicted)
print('target.  : ', y_test)
```

uso il modello addestrato per fare predizioni …

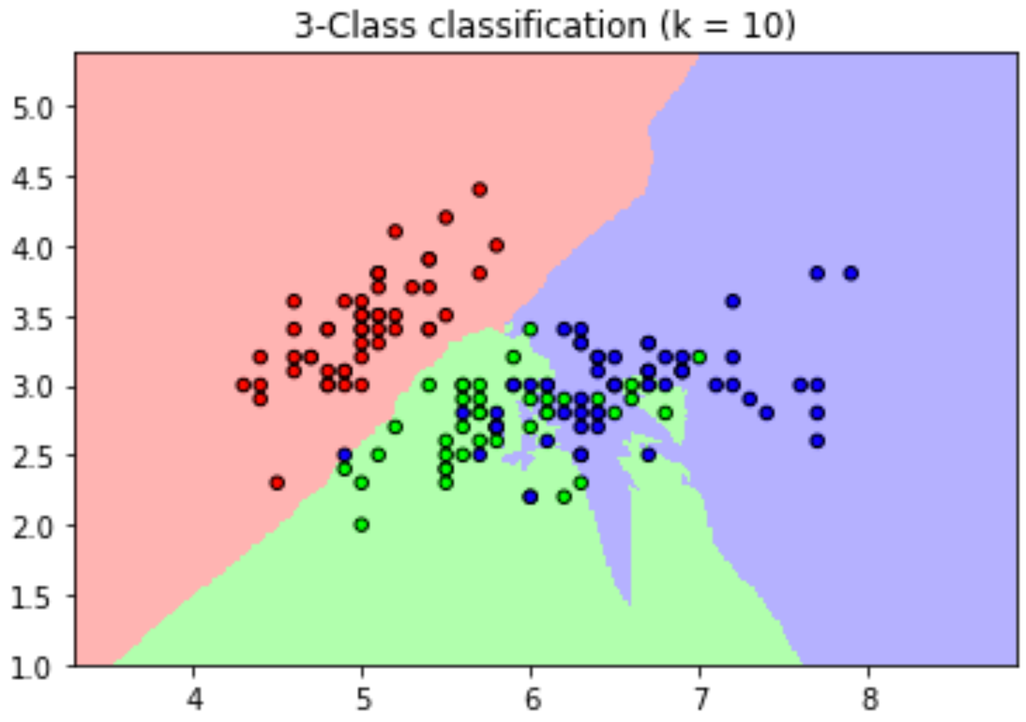
```
predicted:  [1 2 2 0 1 0 0 0 1 2 1 0 2 1 0 1 2 0 1 1 2 2 2 2 2 0 2 2 2 0]
target.  :  [1 1 2 0 1 0 0 0 1 2 1 0 2 1 0 1 2 0 2 1 1 1 1 1 2 0 2 1 2 0]
```

grafico i risultati …

```
# Create color maps per il plot
cmap_light = ListedColormap(['#FFAAAA', '#AAFFAA', '#AAAAFF'])
cmap_bold = ListedColormap(['#FF0000', '#00FF00', '#0000FF'])

h = .02  # step size in the mesh

# Plot the decision boundary. For that, we will assign a color to each
# point in the mesh [x_min, x_max]x[y_min, y_max].
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1

#np.arrange: Return evenly spaced values within a given interval
# np.meshgrid: Return coordinate matrices from coordinate vector
xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                     np.arange(y_min, y_max, h))

# np.c_ : translates slice objects to concatenation along the second axis
Z = clf.predict(np.c_[xx.ravel(), yy.ravel()])

print(Z.shape)

# Put the result into a color plot
Z = Z.reshape(xx.shape)
print(Z.shape)

plt.figure()
plt.pcolormesh(xx, yy, Z, cmap=cmap_light)

# Plot also the training points
plt.scatter(X[:, 0], X[:, 1], c=y, cmap=cmap_bold, edgecolor='k', s=20)
plt.xlim(xx.min(), xx.max())
plt.ylim(yy.min(), yy.max())
plt.title("3-Class classification (k = %i)" % (n_neighbors))
plt.show()
```

```
>>> x = np.array([[1, 2, 3], [4, 5, 6]])
>>> np.ravel(x)
array([1, 2, 3, 4, 5, 6])
```

```
>>> np.c_[np.array([1,2,3]), np.array([4,5,6])]
array([[1, 4],
       [2, 5],
       [3, 6]])
```



ESEMPIO CLASIFICATORI LINEARI CON SCIKIT-LEARN

https://github.com/stefanogiagu/corso_AI_2023/tree/main/notebooks/examples

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression
from sklearn.linear_model import SGDClassifier
from sklearn import datasets
from sklearn.model_selection import train_test_split


# IRIS dataset
iris = datasets.load_iris()
X = iris.data[:, :2]  # take only the first two features
y = iris.target

#select only the first two classes
X = X[y<2]
y = y[y<2]

X_train, X_test, y_train, y_test = \
        train_test_split(X, y, test_size=.5, random_state=21, shuffle=True)
```

IRIS dataset

```
useLogistic = False

if useLogistic == True:
    # C: Inverse of regularization strength (smaller values means stronger regularisation)
    # solver: algorithm to use in the optimization problem (sag: stochastic average gradient)
    logreg = LogisticRegression(C=1.0, solver='sag', max_iter=1000)
else:
   # FDL with loss MSE
   logreg = SGDClassifier(loss='squared_error', max_iter=10000, learning_rate='adaptive', eta0=0.01)

# train the model
_ = logreg.fit(X_train, y_train)
```

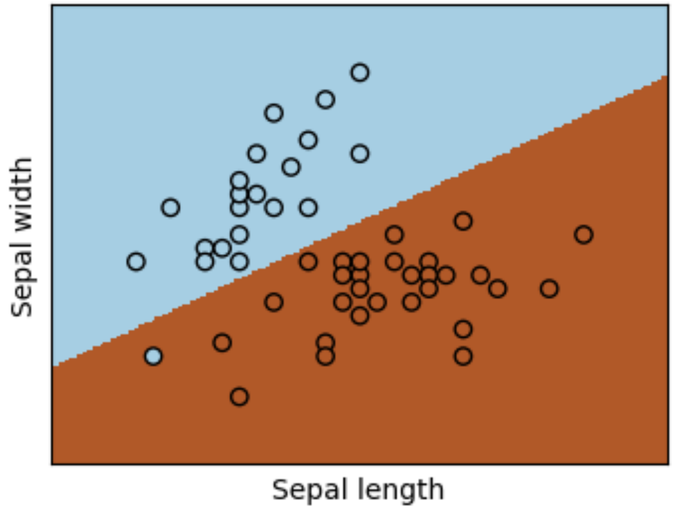
logistic

FDL

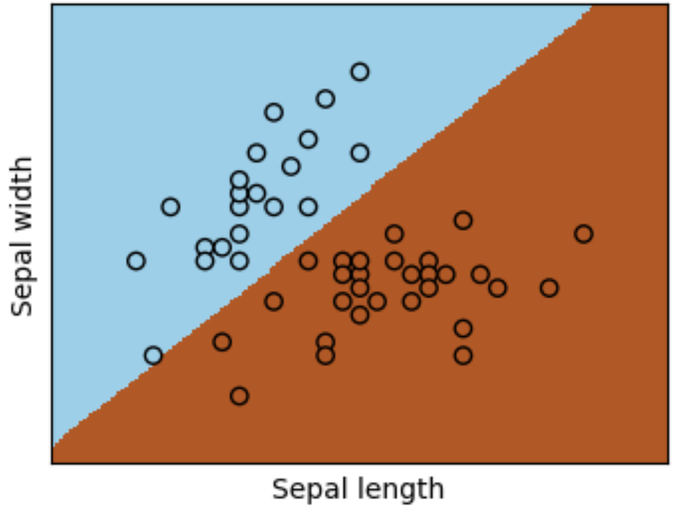
ESEMPIO CLASIFICATORI LINEARI CON SCIKIT-LEARN

```
# Plot the decision boundary. For that, we will assign a color to each
# point in the mesh [x_min, x_max]x[y_min, y_max].
x_min, x_max = X_train[:, 0].min() - .5, X_train[:, 0].max() + .5
y_min, y_max = X_train[:, 1].min() - .5, X_train[:, 1].max() + .5
h = .02  # step size in the mesh
xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))
Z = logreg.predict(np.c_[xx.ravel(), yy.ravel()])

# Put the result into a color plot
Z = Z.reshape(xx.shape)
plt.figure(1, figsize=(4, 3))
plt.pcolormesh(xx, yy, Z, cmap=plt.cm.Paired)

# Plot also the training points
plt.scatter(X_train[:, 0], X_train[:, 1], c=y_train, edgecolors='k', cmap=plt.cm.Paired)
plt.xlabel('Sepal length')
plt.ylabel('Sepal width')

plt.xlim(xx.min(), xx.max())
plt.ylim(yy.min(), yy.max())
plt.xticks(())
plt.yticks(())

plt.show()
```

FDL



logistic



TUTORIALS

- Due testi con elementi ed esempi di uso di numpy, matplotlib, pandas:
- J.VanderPlas, Python Data Science HandBook: https://jakevdp.github.io/PythonDataScienceHandbook/
- W. McKinney, Python for Data Analysis, 2nd Edition: https://wesmckinney.com/book/

- Per imparare features e possibilità d'uso di matplotlib, vi consiglio gli esempi e tutorials disponibili sulla pagina web:
    - https://matplotlib.org/stable/tutorials/index.html
    - in particolare gli introductory tutorials
- Per imparare features e possibilità d'uso di pandas, è più che sufficiente per gli scopi del corso seguirsi il tutoria datacamp:
    - PANDAS_datacamp_tutorial